Docker Vs Traditional Virtualization

In this train, we'll compare Docker with traditional virtualization techniques. We'll identify the key differences and explore the benefits Docker offers over virtual machines.

Learning Objectives

By the end of this train you should be able to:

- Identify the differences and benefits of Docker compared to traditional virtualization techniques.

Virtualization

At its core, virtualization is about abstraction. It's the process of creating a simulated, or virtual, version of something – whether it's hardware, an operating system, or network resources. Think of it as creating a digital replica that behaves like the real thing. This allows us to run multiple isolated environments on a single physical machine, maximizing resource utilization and offering flexibility.

Essentially, virtualization allows us to take a physical server and divide it into multiple virtual servers, each running its own operating system and applications. This has revolutionized how we manage and deploy software.

Traditional Virtualization: Virtual Machines

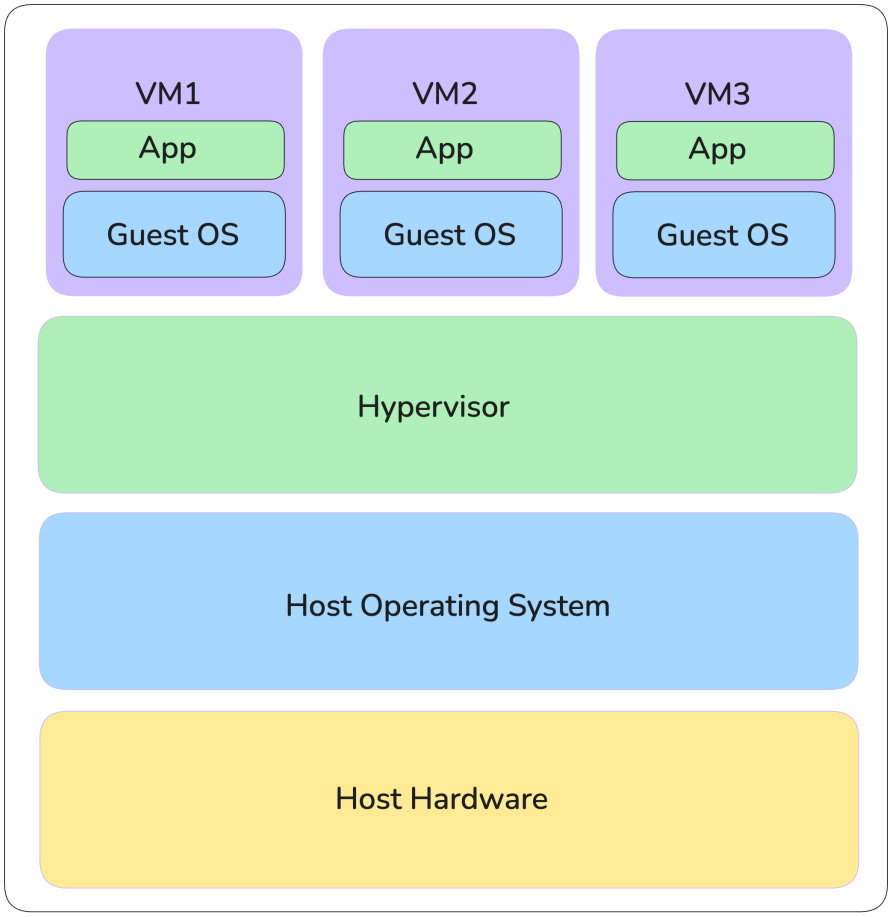


Figure 1: Example of a virtual machine

The most common form of traditional virtualization revolves around virtual machines (VMs). In this model, a hypervisor, such as VMware or Hyper-V, sits between the physical hardware and the virtual machines. The hypervisor creates a complete virtualized hardware layer for each VM, including CPU, memory, storage, and network interfaces.

Each VM then runs its own full-fledged operating system, complete with its own kernel and user space. This provides strong isolation, meaning that if one VM crashes, it won't affect the others. This is incredibly useful for running different operating systems on the same hardware or for isolating critical applications.

What is kernel space? ✖

> In operating systems, kernel space refers to the privileged memory area and processes that constitute the core of the operating system. It's where

What is user space? ✖

> In operating systems, user space refers to the memory area and processes that are accessible and managed by regular user applications. It's

> the kernel itself resides and executes, managing vital resources like the CPU, memory, and hardware devices.

> the part of the operating system where non-kernel code runs.

However, this comes at a cost. Because each VM has its own operating system, they tend to be resource-intensive. They consume significant amounts of CPU, RAM, and disk space, as well as take time to boot up.

> **Reflect:**
> Why would a long boot time be an issue?
> Can you think of a scenario where having a fast boot time is critical for a service?

Docker

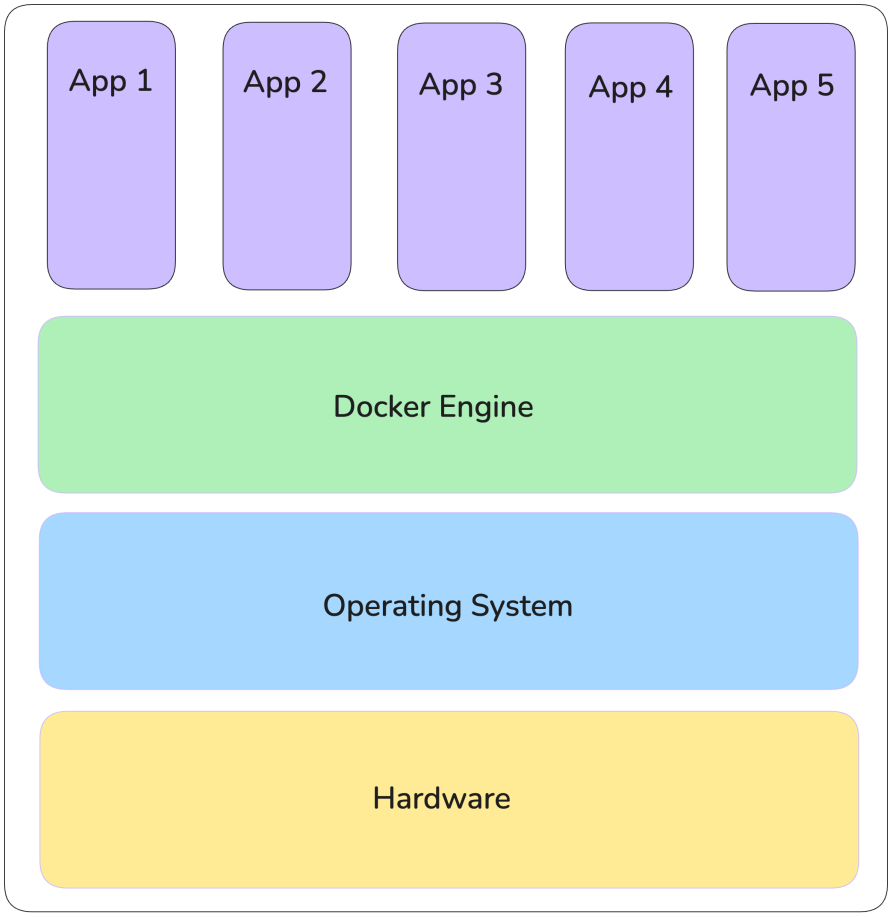


Figure 2: Example of a docker setup

Docker introduces a different approach: **containerization**. Instead of virtualizing the hardware, Docker virtualizes the operating system. This means that multiple containers share the host operating system's kernel, but each container has its own isolated user space.

Think of containers as lightweight packages that bundle an application and its dependencies. Because they don't include an entire operating system, they're significantly smaller and faster than VMs. They start almost instantly and consume far fewer resources.

Key Differences Between Virtual Machines and Containerization

Feature	Virtual Machines (VMs)	Docker (Containers)
Virtualization Type	Hardware Virtualization	OS-level Virtualization

Feature	Virtual Machines (VMs)	Docker (Containers)
Operating System	Each VM has its own guest OS	Containers share host OS kernel
Resource Consumption	High (CPU, RAM, storage)	Low (efficient resource usage)
Boot Time	Minutes	Seconds
Isolation	Hypervisor-level	Process-level
Image Size	Large (gigabytes)	Small (megabytes)
Portability	Portable, but can be hardware-dependent	Highly portable across environments
Hypervisor	Required	Not required
Use Cases	Running different OSs, legacy applications, strong isolation	Microservices, rapid deployment, scalable applications, development environments

Conclusion

In essence, Docker's containerization revolutionizes application deployment by offering a lightweight, efficient alternative to traditional virtual machines. By virtualizing the operating system rather than the hardware, Docker enables faster boot times, reduced resource consumption, and enhanced portability.

Key Takeaways ✖

- **Takeaway 1:** Docker containers provide significant resource efficiency compared to traditional VMs, allowing for higher application density on the same hardware.

- **Takeaway 2:** The speed of container startup and shutdown dramatically improves development and deployment agility.
- **Takeaway 3:** Docker's portability ensures consistent application behavior across diverse environments, simplifying the deployment pipeline.